

Reviewer Pack — Minimal Rank of p-Adic Differentials vs k-SAT Resolution Dep...

Entry 83bbb95eec83 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 03:25:08 UTC

Minimal Rank of p-Adic Differentials vs k-SAT Resolution Depth

Entry ID: 83bbb95eec83 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 03:25:08 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Analysis (Differentials) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for k-SAT

Statement:

[‘For every k-SAT instance with m clauses over n variables, the minimal p-adic order of its differential representation is at most $O(\log(n) + \log(m))$.’, ‘For a given k-SAT instance, if its differential representation has p-adic order greater than $O(\log(n) + \log(m))$, then the resolution proof depth for that instance is at least k^2 .’, ‘This bound holds uniformly for all 30 random seeds of instances.’]

Rationale (proposer’s reasoning):

[‘p-Adic analysis offers a framework to study arithmetic properties of functions, which may uncover hidden complexity-theoretic structures in computational problems like SAT.’, ‘The differential representation of a function captures its local behavior and could be used to measure the difficulty of finding a resolution proof for a SAT instance.’, ‘By studying the relationship between p-adic differentials and resolution depth, we might expose new insights into the nature of SAT complexity.’]

Taxonomy category: p-adic_analysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 31efcb2c45b3f7ac

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all 30 random k-SAT instances, the minimum p-adic order of the differential representation is less than or equal to $\log(n) + \log(m)$, and there is no instance where the p-adic order exceeds $\log(n) + \log(m)$ by more than a factor of 2. The conjecture is falsified if any seed yields an instance with a p-adic order greater than $\log(n) + \log(m)$ or if any instance has a resolution proof depth less than k^2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random k-SAT instance with n variables and m clauses
    n = 20 # Number of variables
    m = min(100, n**2) # Number of clauses (m is at most O(n^2))
    k = 3 # Clause length

    # Create a list of literals (positive for positive, negative for negation)
    literals = [i + 1 if random.choice([True, False]) else -i - 1 for i in range(n)]

    # Create clauses
    clauses = []
    for _ in range(m):
        clause = set()
        while len(clause) < k:
            lit = random.choice(literals)
            if lit not in clause and -lit not in clause:
                clause.add(lit)
        clauses.append(clause)

    # Convert clauses to a list of lists
    clauses_list = [[abs(lit), 1 if lit > 0 else -1] for clause in clauses]

    # Function to compute the minimal p-adic order of the differential representation
    def min_p_adic_order(clauses):
        # Placeholder implementation (this is where you would implement actual p-adic computation)
        return max(len(clause) for clause in clauses)
```

```

# Compute the minimal p-adic order
p_adic_order = min_p_adic_order(clauses_list)

# Function to compute the resolution proof depth using DPLL algorithm (simplified version)
def dpll(clauses, assignment):
    if not clauses:
        return 0
    unit_clause = next((c for c in clauses if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        return dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment)
    pure_literal = next((l for l in range(1, n+1) if (l in [c[0] for c in clauses] and -l not in [c[0] for c in clauses])), None)
    if pure_literal:
        new_assignment = assignment.copy()
        new_assignment[pure_literal] = True
        return dpll([c for c in clauses if pure_literal not in c and -pure_literal not in c], new_assignment)
    literal = random.choice([l for l in range(1, n+1) if l not in assignment])
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    return dpll(clauses_list, new_assignment) + 1

# Compute the resolution proof depth
resolution_depth = dpll(clauses_list, {})

# Check the conjecture
conjecture_holds = p_adic_order <= math.log(n, 2) + math.log(m, 2)
counterexample = "" if conjecture_holds else f"p-adic order {p_adic_order} exceeds log(n) + log(m)"

return {
    "metric_name": "Minimal p-adic Order vs Resolution Depth",
    "metric_value": p_adic_order,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 53)) # Default seeds

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])

```

```

    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\") first_failing_seed={seed}
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59dca5f5.py", line 91, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59dca5f5.py", line 71, in run_trial
    resolution_depth = dpll(clauses_list, {})
                      ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59dca5f5.py", line 68, in dpll
    return dpll(clauses_list, new_assignment) + 1
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59dca5f5.py", line 68, in dpll
    return dpll(clauses_list, new_assignment) + 1
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59dca5f5.py", line 68, in dpll
    return dpll(clauses_list, new_assignment) + 1
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 17 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59dca5f5.py", line 65, in dpll
    literal = random.choice([l for l in range(1, n+1) if l not in assignment])
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')
IndexError: Cannot choose from an empty sequence

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the pre-registered support conditions could not be verified. | next: Re-run the test to verify the conjecture under the specified conditions.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12298
2	propose	ollama_remote	glm4:latest	0	0	12797
3	preregistration	ollama_remote	glm4:latest	0	0	6705
4	novelty	ollama_remote	glm4:latest	0	0	4731
5	novelty	ollama_remote	glm4:latest	0	0	5096
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25305
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10391
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9894
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12555
10	judge	ollama_remote	glm4:latest	0	0	10845

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110615 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/83bbb95eec83.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/83bbb95eec83.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/83bbb95eec83.tar.gz* (if generated)